Applications & Paradigms

Part 1

Anandha Gopalan
(with thanks to Teo Yong Meng)

axgopala@nus.edu.sg

NUS National University of Singapore

Outline

Cloud Applications

Cloud Application Development Models

Recall: Cloud Providers

Common Features

Basic and higher-level services with different levels of abstraction (IaaS, PaaS, SaaS)

Compute and storage resources – virtual servers, storage, networking, ...

Deployment management services – load balancer, auto-scaling, monitoring, message queueing, ...

for computational resources

User interface – graphical UI, CLI

...

NUS National University of Singapore

Cloud Applications

Focus primarily on enterprise computing

Ideal ☺

Applications which can be divided into as many independent tasks as possible

E.g., Web services, database services, search services, machine learning with large-scale models, ...

Not Ideal

Applications with a complex workflow and multiple dependencies, such as high-performance computing

e.g. part A -> part B -> part C

Applications with intense communication among concurrent instances

networking issues

Workload cannot be arbitrarily partitioned

bandwidth and latency

Key Challenges!

Cloud Consumer – scale application to accommodate a dynamic load, recover after system failure, efficiently support checkpoint/restart, ...

Cloud Provider – manage a large number of systems, provide quality of service guarantee, ...

Performance Isolation

Virtualisation shares VMs and multitenancy shares cloud application services ⇒ problem when utilisation is high!

multiple copies at different locations

Reliability

Server failures expected when a large number of servers cooperate to compute

Latency/Bandwidth Fluctuations

Due to sharing of resources

Data Logging

Necessary, but also need to limit it due to performance considerations

way to recover -> backups
how often? you'll need to interrupt your app

Reliance on Internet and web technology – high accessibility, web browser universality, ease of web-based service development

Cloud services use web technology as both the implementation medium as well as the management interface

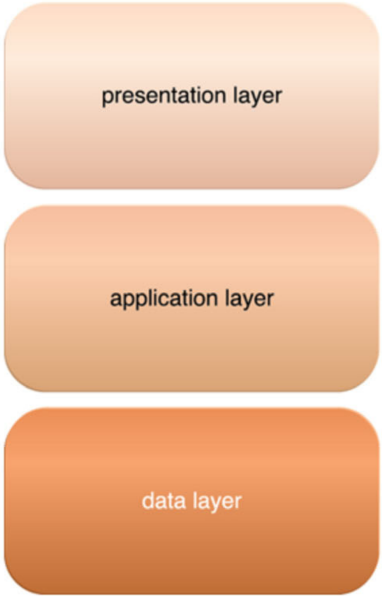
Cloud applications typically use a web-based client-server architecture – web browser client and web server

API - 2 elements are communicating
Some kind of contract between

Architecture of Web Applications I

Three-tier architecture abstraction for a web application

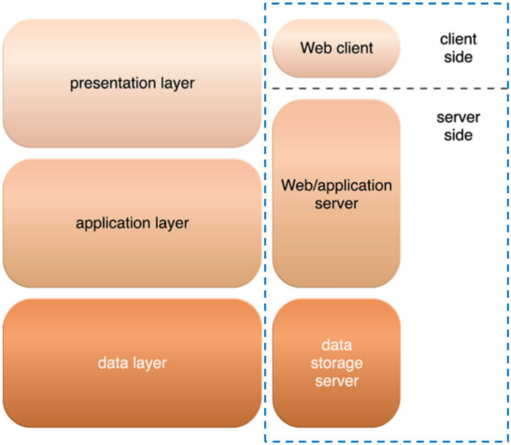
1. **Presentation layer** – represents user-interface
2. **Application layer** – represents implementation logic
3. **Data layer** – comprises of persistent data stores



NUS National University of Singapore

Architecture of Web Applications II

1. **Presentation layer** – sits on both the client and server side
2. **Application layer** – Web server receives client requests and retrieves the requested resources or generates web content based on application logic
3. **Data layer** – Web servers interact with application servers and the underlying database(s)



Web services technology – provides "as-a-service" cloud delivery models

Simple Object Access Protocol (SOAP)

Application protocol for web applications

Defines a common web service messaging format for request and response exchanges – based on XML and uses TCP or UDP as the transport protocols

protocol - set of rules defining communication btw 2 parties

structured format, larger enterprises, security important

Architecture of Web Applications IV

Representational State Transfer (REST)

Software Architecture for distributed hypermedia systems

Supports client communication with stateless servers

Platform and language independent

Supports data caching

lightweight option

hypermedia - different types of media

Uniform interface

Client-server decoupling

Statelessness — when you connect again, the system has no knowledge of what happened previously

Cacheability — so you can return repeated requests faster

Layered system architecture

Code on demand (optional)

Web Services for Cloud Applications

Cloud services are naturally distributed – single service request might require message passing between multiple servers

Cloud services are accessed by different clients using different programming languages and technologies (e.g., Java, Python, ...) – requires a standard interface for communication between clients and server `language agnostic`

Web service APIs implement a standard communication interface for cloud services, e.g., REST API requests/responses are transferred on HTTP using formats such as XML and JSON

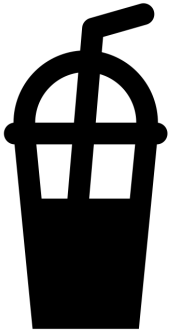
NUS National University of Singapore

Cloud Application Development Models

Characteristics of Cloud Service Models

Example of setting up a data analytics application

Take a Break

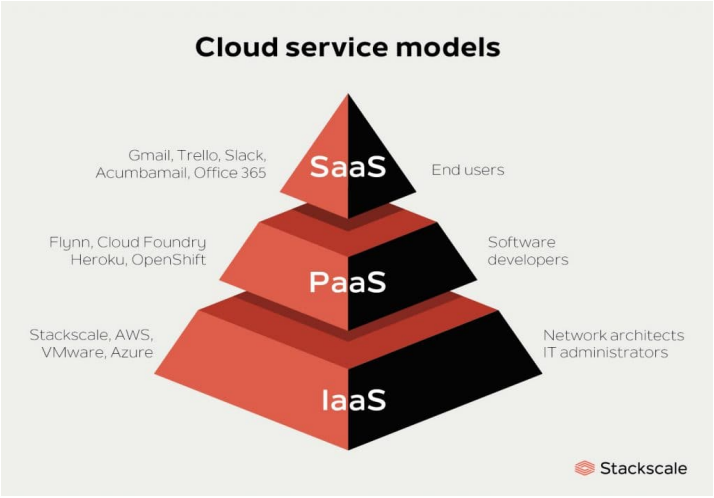






NUS National University of Singapore

Recall: Basic Cloud Service Models

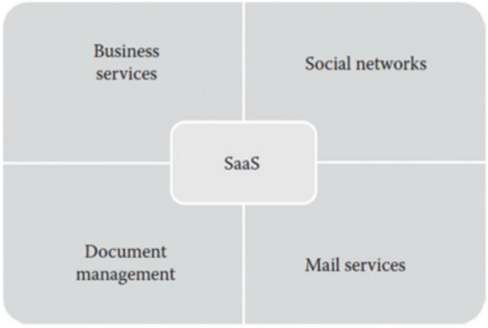


https://www.stackscale.com/wp-content/uploads/2020/04/cloud-service-models-iaas-paas-saas-stackscale-1024x711.jpg

Software as a Service

Consumers execute developed applications by supplying data and obtaining the results

Provider develops and provides the application – running of application is transparent to the consumer



Services Provided by SaaS Providers

NUS National University of Singapore

SaaS Providers

Characteristics

Multi-tenanted applications

Web access

Centralised management of SaaS services

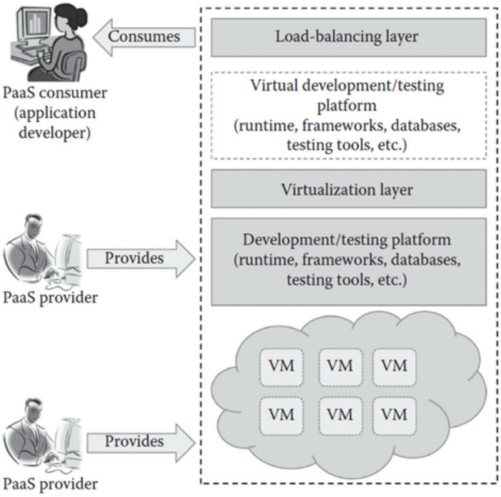
Multi-device support

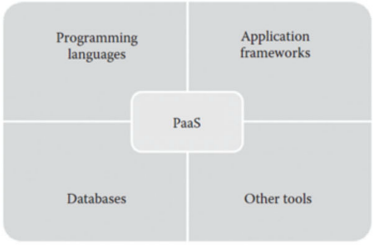
Scalability under varying loads

High availability

API integration with other software

Platform as a Service





Services Provided by PaaS Providers

used by application developers

Overview of PaaS

PaaS Providers

Characteristics

All in one – same IDE to develop, test, deploy, host, and maintain applications

Web access to development platforms

Offline access for developers

Built-in scalability

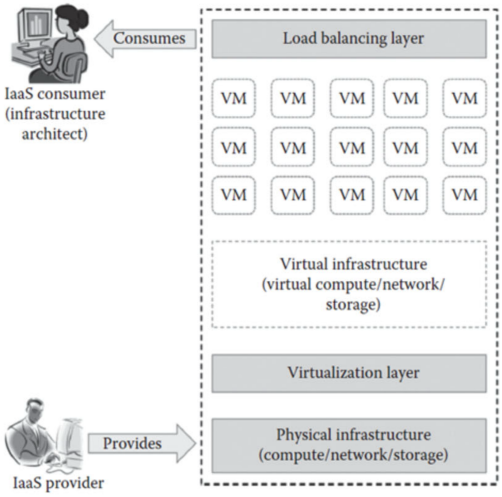
Collaborative platform for developers

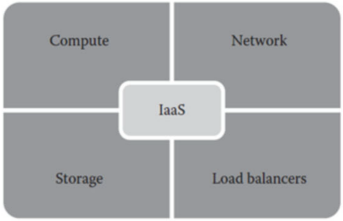
Diverse client tools

examples: Salesforce, Heroku, Digital Ocean

NUS National University of Singapore

Infrastructure as a Service





Services Provided by IaaS Providers

Overview of IaaS

IaaS Providers

Characteristics

Web access to resources — `LAMP stack - packaged dev env`

Centralised physical resource management

Elastic services and dynamic scaling

Shared infrastructure across multiple users

Precofigured VMs

Metered services

NUS National University of Singapore

Setting up a Data Analytics Application

Key Question!

Which cloud service model to use, i.e., which "as-a-service" do you go for?

... It depends! ☺

Application is located on the cloud

Clients use a thin client (web browser) to access application

Often built on top of multi-tiered PaaS and IaaS

- Clients do not deal with underlying PaaS and IaaS
- Clients do not deal with APIs

Various pricing mechanisms – free!, pay-as-you-go, subscription, etc.

Use a data analytics application – SaaS consumer

Key Steps

1. Access your favourite application or AI ☺
2. Give it your data
3. Report on the analysis!

Setting up a Data Analytics Application

PaaS

Consumer

Creates software using tools and libraries from PaaS provider

Controls software deployment and configuration settings

Provider

Installs and maintains software, libraries and middleware `keep things up to date`

Handles the transparent infrastructure from the user

Exposes an API for PaaS consumers (programmers)

Provides integrated services of scalability, maintenance, and versioning, e.g., total application hosting, development, testing, and deployment environment

Set up a data analytics application – PaaS consumer

Key Steps — `package to build web application`

1. Access a system with MongoDB, Express, React, Node.js (MERN)
2. Build your own data analytics pipeline
3. Give it your data
4. Report on the analysis!

Set up a data analytics application – IaaS consumer

Key Steps

1. Create a compute instance running Linux
2. Install and configure MongoDB, Express, React, Node.js (MERN)
3. Build your own data analytics pipeline
4. Give it your data
5. Report on the analysis!

IaaS	PaaS	SaaS
Create a compute instance running Linux	...	...
Install and configure MongoDB, Express, React, Node.js (MERN)	Access a system with MongoDB, Express, React, Node.js (MERN)	...
Build your own data analytics pipeline	Build your own data analytics pipeline	Access your favourite application or AI
Give it your data	Give it your data	Give it your data
Report on the analysis	Report on the analysis	Report on the analysis

Next Week

Recess Week ♫

Week After

Applications and Programming (Part 2)

NUS National University of Singapore

References

REST APIs (`https://www.ibm.com/think/topics/rest-apis`)

Continuous Feedback



https://forms.cloud.microsoft/r/ZJxTf2wFjr

NUS National University of Singapore